



ESKISEHIR OSMANGAZI UNIVERSITY
ELECTRICAL AND ELECTRONICS ENGINEERING
2021-2022 SPRING SEMESTER
DIGITAL SYSTEMS LABORATORY

EXPERIMENT 7
INTRODUCTION TO LOGIC DESIGN WITH XILINX ISE



MUSTAFA YAĞCI
151220192027
GROUP II – D



EXPERIMENT 7 REPORT

OBJECTIVES

- The aim of this experiment is to familiarize ourself with logic design using the Xilinx software.

INTRODUCTION

Xilinx tools make it possible to create schematics, hardware description languages, and custom modules in a number of ways. This lab demonstrates how to enter a combinational logic circuit's verilog overview and test your HDL specification in Xilinx.

PROCEDURE

1- Firstly, follow the steps to setup Xilinx program into your computer. Then, create a new Xilinx project (its name will be your name and surname in format: AhmetDemir) in your computer. Follow the Step 1, Step 2 and Step 3 in your lab manual and do all the things in these steps. Add the photos of the written codes for steps 2 and 3 (Added photos must contain the project name like the photos given in Figure 4 and Figure 8 in your lab manual) into your report. Moreover, add the simulation graph output obtained in Step 3 into your report.

Hint: Do not use any Turkish character in project name and use “D:\” folder for project location.

```

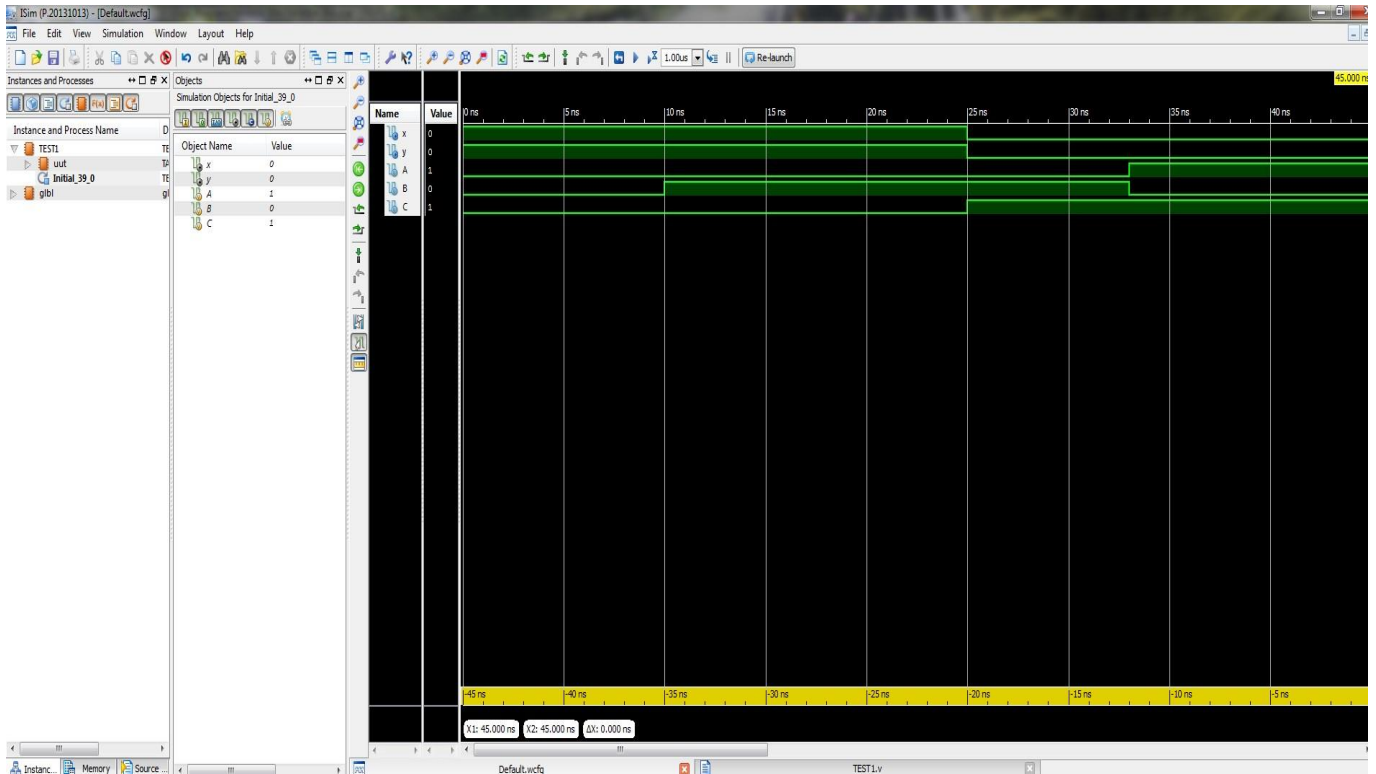
1 `timescale 1ns / 1ps
2 ///////////////////////////////////////////////////////////////////
3 // Company:
4 // Engineer:
5 //
6 // Create Date:    14:26:22 05/15/2022
7 // Design Name:
8 // Module Name:
9 // Project Name:   Mustafa Yağcı
10 // Target Devices:
11 // Tool versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21 module MustafaYagci (A,B,C,x,y
22 );
23 input A,B,C;
24 output x,y;
25 wire w1;
26
27 and G1(w1,A,B);
28 not G2(y,C);
29 or G3(x,w1,y);
30
31
32 endmodule
33

```

```

14 //
15 // Verilog Test Fixture created by ISE for module: Mustafa Yağcı
16 //
17 // Dependencies:
18 //
19 // Revision:
20 // Revision 0.01 - File Created
21 // Additional Comments:
22 //
23 ///////////////////////////////////////////////////////////////////
24
25 module TEST1;
26
27 // Inputs
28 reg A;
29 reg B;
30 reg C;
31
32 // Outputs
33 wire x;
34 wire y;
35
36 // Instantiate the Unit Under Test (UUT)
37 MustafaYagci uut (A,B,C,x,y);
38
39 initial begin
40 // Initialize Inputs
41 A = 1'b0; B = 1'b0; C = 1'b0;
42 // Wait 10 ns
43 #10;
44 A = 1'b0; B = 1'b1; C = 1'b0;
45 // Wait 15 ns
46 #15;
47 A = 1'b0; B = 1'b1; C = 1'b1;
48 // Wait 8 ns
49 #8;
50 A = 1'b1; B = 1'b0; C = 1'b1;
51 // Wait 12 ns
52 #12;
53 $finish;
54 // Add stimulus here
55
56 end
57
58 endmodule
59
60

```



DISCUSSION AND CONCLUSIONS

This experiment demonstrates how to use logic design codes in computer software. The process began after the required program was installed. The design was created using Verilog source. Inputs and outputs are defined as modules in the coding section. There are also cables and gates listed. The keywords are blue, the comment lines are orange, the data is red, and the values are black. After that, a new project was created for the simulation case, with Verilog Test Fixture as the source. The required coding procedures have been completed. The coding process was saved after it was over, and the simulation phase began. Importing our previously prepared code kicked off the simulation process. The simulation results were mirrored on the screen as a result.